

cTEE: Cryptographic Trusted Execution Envelopes

Private Workspaces and Protected Agency on Untrusted Compute

cTEE Yellow Paper

June 1, 2026

Abstract

Modern agent harnesses increasingly need persistent remote compute: rented GPUs, hosted workers, decentralized compute nodes, service marketplaces, and always-on cloud workspaces. The standard deployment model gives the remote node a plaintext workspace and then asks containers, VMs, process sandboxes, or boot measurements to keep a root-level provider from reading it. That model is wrong for high-value agent state. A provider with root can usually inspect files, logs, process memory, prompts, tool inputs, and GPU-visible buffers.

This paper introduces cTEEs, Cryptographic Trusted Execution Envelopes, as a software systems pattern for private autonomous agents on untrusted compute. A cTEE is not a hardware TEE and does not claim to hide arbitrary plaintext memory from root. Its central invariant is stricter: protected workspace state is not given to the untrusted node as normal plaintext in the first place. The user-facing primitive is an *Open Private Workspace*; the node-facing primitive is a cryptographic execution envelope over encrypted workspace objects, public trunks, sealed private heads, commitments, receipts, leakage profiles, declassification gates, and capability exits. A cTEE can be bound to any private agent harness, enterprise runner, DePIN network, worker runtime, or service marketplace that can provide an authority view, execution boundary, state log, artifact store, and optional settlement layer.

The paper contributes: (1) a formal no-plaintext-custody model for private agent workspaces; (2) a concrete object and protocol suite including `PrivateWorkspaceNode`, `PrivateWorkspaceCapsule`, `AlphaSeal`, `AutonomyLease`, `DeclassificationReceipt`, and `PrivateInferenceReceipt`; (3) a compositional security treatment for privacy, authority, restore validity, and receipt binding; and (4) a practical default high-throughput execution strategy, Candidate-Lattice Private Decoding, that keeps ordinary GPU token speed for public model work while moving private selection, private policy, or private strategy heads outside the untrusted node. It also defines Plaintext-Free Runtime Mounting, including Plaintext-Free Model Mounting, as the runtime interface that exposes private workspaces to runtimes as public/redacted content, encrypted refs, private handles, and capability exits rather than provider-readable plaintext. We also state a negative result: arbitrary full-speed private LLM inference on a single root-controlled consumer GPU is not obtained by software wrapping alone. Useful privacy requires either no plaintext custody, trusted hardware, a trusted or threshold guardian, or cryptographic evaluation with its own overhead. cTEEs make that tradeoff explicit and productizable.

Contents

1	Motivation	4
1.1	Why This Is Not Encryption at Rest	4
2	Contributions	4
2.1	Security Claims and Non-Claims	5

3	System Model	5
3.1	Participants	5
3.2	Guardian as Authority View	6
3.3	Architecture at a Glance	7
3.4	Reference Bindings	7
3.5	Adversary	8
3.6	State Partition	8
4	Cryptographic Preliminaries	9
4.1	Authenticated Encryption	9
4.2	Commitments	9
4.3	Homomorphic Encryption, MPC, Garbled Circuits, and ORAM	9
4.4	Receipts	9
5	cTEE Syntax	10
6	Leakage-Typed cTEE-IR	10
6.1	Sensitivity Lattice	10
6.2	Remote Admissibility	11
6.3	Receipt-Carrying Types	11
6.4	Compiler Invariant	12
7	Core Objects	12
7.1	PrivateWorkspace	12
7.2	PrivateWorkspaceCapsule	12
7.3	AlphaSeal	13
7.4	AutonomyLease	13
7.5	NodeMeasurementReceipt	14
8	Private Workspace File System	14
8.1	Encrypted Names and Patches	14
8.2	Public Trunk and Private Head	15
9	Plaintext-Free Runtime Mounting	15
10	Deterrence and Detection	16
11	Protocols	17
12	Security Definitions	18
12.1	No-Plaintext-Custody	18
12.2	Authority Safety	18
12.3	Receipt Binding	19
12.4	Restore Validity	19
13	Security Theorems	19

14 Candidate-Lattice Private Decoding	20
14.1 Construction	20
14.2 Why It Matters	20
14.3 Security	21
14.4 Selection Leakage Accounting	21
14.5 Agent Use Cases	22
15 Default cTEE Agent Execution	22
16 Quantitative Strategy Pattern	22
16.1 End-to-End Quant Walkthrough	23
17 Leakage Profiles	24
17.1 Leakage Receipts	24
18 Implementation Guidance	25
18.1 Harness Integration	25
18.2 Minimum Viable cTEE	25
18.3 Conformance Tests	26
19 Relation to Existing Work	26
20 Open Problems	26
21 Conclusion	27

1 Motivation

Private autonomous agents need compute that can stay awake. A developer may want to rent a cheap RTX 3090 node to run a persistent agent harness. A quant user may want the node to fetch market data, run backtests, simulate models, draft reports, and propose trades while the user’s laptop is asleep. An enterprise may want a service marketplace to spin up decentralized compute for a private outcome. In each case the economic reason to use the node is clear: GPU time is cheaper or more available away from the user’s local machine.

The privacy problem is equally clear. If the remote workspace contains:

```
/workspace/strategy.py  
/workspace/signals.py  
/workspace/risk.py  
/workspace/config.yaml
```

as ordinary plaintext, a root-level provider can likely copy the files, snapshot the disk, trace the process, inspect logs, and watch prompts. An encrypted disk image helps at rest, but once the guest mounts the volume and the agent reads the strategy, the provider-controlled host can often observe the plaintext through some layer of the system. A container boundary is not a privacy theorem.

cTEE doctrine. The node can compute against the workspace, but it should not custody workspace secrets.

1.1 Why This Is Not Encryption at Rest

Encryption at rest protects stored bytes until a program decrypts them. It does not, by itself, protect the plaintext once the remote workspace mounts the volume, prompts a model, runs a tool, writes a log, caches a result, or places intermediate values in provider-visible memory.

cTEE changes the routing contract. Protected state is not mounted, prompted, logged, cached, traced, or executed as provider-readable plaintext on the untrusted node. The node receives public trunks, redacted projections, encrypted refs, commitments, candidate lattices, and receipts. Plaintext views and declassification decisions live in the user’s authority view.

cTEEs are proposed as the missing middle between two bad extremes:

1. **Plain cloud workspace.** Fast and useful, but the provider can read the secrets.
2. **Pure cryptographic private inference.** Stronger privacy, but today usually too slow or too narrow for arbitrary agentic LLM work.

cTEEs keep the useful fast path by splitting work into public or redacted trunks that can run on the rented GPU, and private heads that remain sealed, masked, secret-shared, guardian-mediated, or locally evaluated.

2 Contributions

This yellow paper defines cTEEs as a systems architecture rather than as a new primitive. Its technical contribution is the envelope and composition model:

1. **No-Plaintext-Custody Security.** We formalize the property that an untrusted node’s view is simulatable from declared leakage, public trunks, encrypted refs, commitments, and receipts.

2. **Private Workspace Object Model.** We define the objects needed to make “Open Private Workspace” real: encrypted workspace objects, private workspace capsules, sealed private heads, autonomy leases, declassification receipts, and capability exits.
3. **Split-State Agent Automaton.** We model agent execution as a state machine whose state is partitioned into public, redacted, sealed, guardian, and action-authority components.
4. **AlphaSeal.** We define a reusable sealed-private-head pattern for quant strategies and other high-value decision logic.
5. **Candidate-Lattice Private Decoding.** We introduce the default cTEE protected-agency strategy: an untrusted GPU generates public candidate lattices at normal speed while a private head selects, filters, or reranks without revealing the private policy.
6. **Plaintext-Free Runtime Mounting.** We define the runtime mount boundary that presents private workspaces to tools, files, shells, model servers, and model calls as public trunks, redacted projections, encrypted refs, private-function handles, declassification requests, and capability exits rather than plaintext private files or prompts. `PlaintextFreeModelMount` is the model-facing specialization.
7. **Deterrence and Detection.** We specify synthetic canaries, provider-bound watermarks, honeytokens, leak scans, replay checks, and receipts as attribution and dispute evidence. These mechanisms do not make plaintext safe; they make abuse more detectable.
8. **Receipt and Restore Composition.** We specify how receipts, commitments, encrypted blobs, and state roots make private agent work auditable and restorable without exposing plaintext to the compute provider.

2.1 Security Claims and Non-Claims

Claims	The node does not receive plaintext custody of protected workspace objects. Private objects are encrypted, committed, redacted, guardian-mediated, or handled through explicit capability exits. External effects require authority and receipts. Restore/import is state-root and policy checked.
Non-claims	cTEE does not claim arbitrary full-speed private LLM inference by software wrapping alone. It does not eliminate all metadata, timing, size, schedule, or selection leakage. It does not protect a workspace that the user explicitly mounts on the rented node as ordinary plaintext.

3 System Model

3.1 Participants

A cTEE deployment has the following roles:

USER. The human, organization, or local client that owns private workspace state.

NODE. The untrusted rented, hosted, provider, or DePIN compute node. It may have a GPU and may run an agent harness. It is assumed provider-rooted.

GUARDIAN. A trusted, threshold, or policy-controlled authority path that can hold keys, evaluate narrow private heads, approve declassification, or sign capability exits. This may be a browser/device, mobile signer, enterprise key service, MPC committee, HSM, or confidential environment.

STATELOG. The append-only or operation-backed state log that records workspace refs, commits, receipts, state roots, policy hashes, and restore metadata. The state log is the canonical meaning layer for sealed objects, not the place where all payload bytes must live.

BLOBSTORE. The blob storage plane. It holds encrypted payload bytes, sealed archives, traces, checkpoints, public datasets, and large artifacts. It does not define what those bytes mean.

3.2 Guardian as Authority View

The guardian is a formal role, not necessarily a separate machine the user must operate. In the common product path, the guardian is the user's existing authority view:

- an authenticated browser instance that renders decrypted private files;
- a desktop agent harness or CLI holding an interactive workspace session;
- an authority plane or signing path backed by passkeys, hardware keys, mobile approval, policy services, or threshold custody;
- a customer VPC service, enterprise HSM, or policy service for teams;
- a small always-on authority worker for unattended `AutonomyLease` checks.

Thus `cTEE` does not require a second runtime beside the rented GPU node. It separates compute custody from view/sign/declassify custody. The user already needs some surface that can decrypt and display private workspace content; the guardian names that surface when it also evaluates private heads, applies declassification policy, or approves capability exits.

Authority-view doctrine. The rented node is the compute workspace. The browser, authority plane, local harness, CLI, or threshold authority is the private view and signing membrane.

3.3 Architecture at a Glance

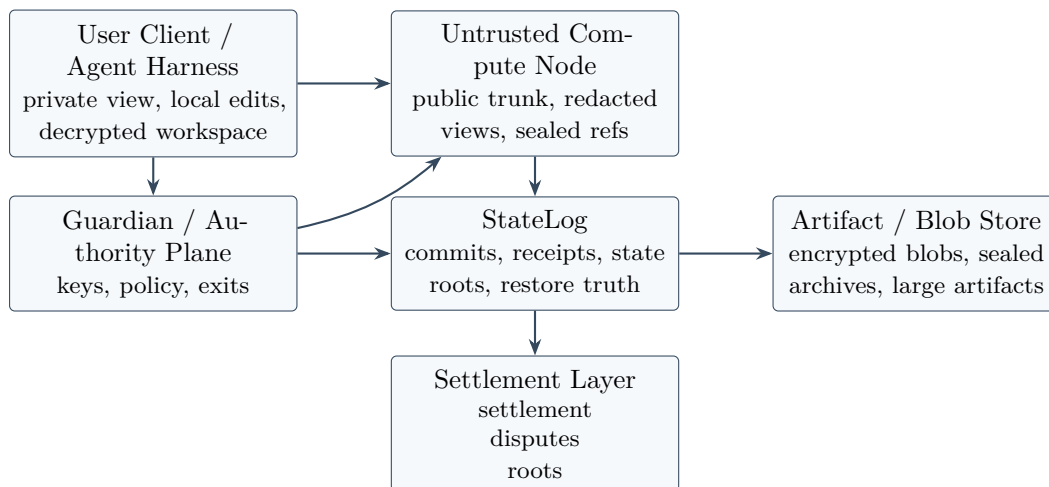


Figure 1: cTEE splits plaintext custody from compute custody. The rented node receives useful work but not protected workspace plaintext. The state log records meaning, receipts, policy, and restore truth; artifact stores hold bytes; public settlement is sparse and trigger-based.

3.4 Reference Bindings

cTEE is ecosystem-neutral. A deployment binds the generic roles above to its own runtime, authority, state, storage, marketplace, and settlement components. The protocol does not require Agentgres, wallet.network, IOI L1, or any particular marketplace. Those names are one concrete IOI binding:

Guardian / authority view.

Authenticated browser, local Autopilot, CLI signer, wallet.network-backed authority.

Authority plane.

wallet.network for grants, scopes, revocation, secrets, payments, and decryption leases.

Execution boundary.

IOI daemon for tool/model execution, sensitive-exit trapping, and runtime mount enforcement.

Plaintext-Free Runtime Mount.

Default Harness Profile cTEE mount inside the daemon.

StateLog.

Agentgres for admitted refs, receipts, state roots, policy hashes, and restore truth.

Artifact / blob store.

Agentgres Artifact Plane plus storage backends such as local disk, S3, Filecoin, CAS/IPFS, and object stores.

Settlement layer.

IOI L1 or compatible app chains for public, economic, dispute, or cross-domain commitments only when triggered.

Worker marketplace.

aiagent.xyz for discovery, manifests, benchmarks, reputation, and licensing. It advertises capabilities; it does not grant capabilities.

Outcome marketplace.

sas.xyz for service ordering, delivery, acceptance, dispute, remediation, and settlement profile. Authority still exists through the daemon and wallet.network.

Binding doctrine. cTEE defines the privacy and custody envelope. A concrete stack binds that envelope to an authority plane, execution boundary, state log, artifact store, and settlement layer. Marketplaces may advertise or contract for capabilities and outcomes, but they do not grant capabilities.

3.5 Adversary

The adversary controls NODE. It can:

- read node files, logs, environment variables, process memory, and normal GPU-visible buffers;
- snapshot disks and RAM;
- observe network timing, job sizes, model family, public datasets, and resource usage;
- delay, abort, replay, fork, or equivocate about node execution;
- return invalid outputs or try to get paid for fake work.

The adversary does not break standard cryptographic assumptions, does not compromise GUARDIAN unless the chosen guardian threshold permits it, and does not force the user to explicitly declassify protected state. Hardware TEEs may be used as an optional strengthening layer, but the base cTEE model does not depend on them.

3.6 State Partition

Let an agent workspace state S be partitioned as:

$$S = S_{\text{pub}} \parallel S_{\text{red}} \parallel S_{\text{seal}} \parallel S_{\text{guard}} \parallel S_{\text{auth}}.$$

- S_{pub} : public files, public model weights, public data, public tools.
- S_{red} : redacted summaries, public stubs, sanitized prompts, padded labels.
- S_{seal} : encrypted private workspace objects, sealed strategy heads, encrypted archives, commitments.
- S_{guard} : state available only to GUARDIAN or a threshold committee, such as key shares, private coefficients, private selectors, or declassification policy.
- S_{auth} : action authority, including signing keys, broker capability scopes, payment authority, and revocation state.

The core cTEE invariant is:

$$\text{View}_{\text{Node}} \subseteq f(S_{\text{pub}}, S_{\text{red}}, \text{Enc}(S_{\text{seal}}), \text{Com}(S_{\text{guard}}), \mathcal{L})$$

for declared leakage $\mathcal{L}$. The node’s view must not require plaintext S_{seal} , S_{guard} , or S_{auth} .

4 Cryptographic Preliminaries

4.1 Authenticated Encryption

Workspace objects are encrypted with an authenticated encryption scheme:

$$C \leftarrow \text{AEAD.Enc}_K(N, A, M), \quad M \leftarrow \text{AEAD.Dec}_K(N, A, C).$$

The associated data A should bind workspace identity, object id, schema version, policy hash, content role, state root, and expected parent head. This prevents ciphertext transplant across workspaces or policy epochs.

4.2 Commitments

A commitment scheme provides hiding and binding commitments. For scalar or vector values, a Pedersen-style commitment has the form:

$$C = rH + \sum_i v_i G_i,$$

where r is a random blinding factor and G_i, H are independent generators. It is additively homomorphic:

$$C(\mathbf{v}, r) + C(\mathbf{w}, s) = C(\mathbf{v} + \mathbf{w}, r + s).$$

This is useful in cTEEs for private scores, private risk budgets, aggregate usage, masked rankings, and proof-carrying receipts. The commitment is not a substitute for encryption; it is a public binding to a private value.

4.3 Homomorphic Encryption, MPC, Garbled Circuits, and ORAM

Fully homomorphic encryption (FHE) evaluates circuits over ciphertexts. Approximate arithmetic schemes such as CKKS are relevant for vector and neural arithmetic, but have cost, precision, and circuit-depth constraints. Secure multi-party computation (MPC) distributes computation across parties such that privacy holds against a bounded collusion threshold. Garbled circuits are often effective for boolean or branching logic. Oblivious RAM (ORAM) hides access patterns at a bandwidth and complexity cost.

cTEEs treat these as operator families:

$$\text{Op} \in \{\text{Public}, \text{Redacted}, \text{FHE}, \text{MPC}, \text{GC}, \text{ORAM}, \text{Guardian}, \text{TEE}\}.$$

The envelope chooses the cheapest operator family that satisfies the sensitivity policy. It does not require every operation to be FHE.

4.4 Receipts

A receipt is a signed or hash-addressed statement that binds:

$$R = H(\text{actor}, \text{operation}, \text{inputs}, \text{outputs}, \text{policy}, \text{authority}, \text{stateRoot}, \text{time}).$$

Receipts let the system prove that an operation occurred under a policy without revealing protected payloads. A receipt may include commitments instead of plaintext outputs.

5 cTEE Syntax

A cTEE scheme is a tuple of algorithms:

$$\Pi_{\text{cTEE}} = (\text{Setup}, \text{Open}, \text{Seal}, \text{Patch}, \text{Compile}, \text{Eval}, \text{Declassify}, \text{Exit}, \text{Commit}, \text{Restore}).$$

Setup(1^λ). Generates system parameters, encryption profiles, commitment parameters, supported operator families, and receipt schemas.

Open(User, P). Opens a private workspace under policy P . The user sees ordinary files and folders; the node sees only permitted views.

Seal(W, M, role). Encrypts a workspace object M into a sealed object C with role metadata, state roots, and policy-bound AAD.

Patch(W, Δ). Converts a user edit into an encrypted patch, a public trunk patch, or a redacted projection update.

Compile(task, P). Compiles a task into a **PrivateWorkspaceCapsule**: visible context, protected refs, allowed remote ops, forbidden plaintext classes, leakage profile, and required receipts.

Eval($\text{Node}, \text{capsule}$). Executes public, redacted, masked, encrypted, or guardian-mediated operations and emits commitments and receipts.

Declassify(Y, P, grant). Converts a protected output into a visible result or denies/escalates it.

Exit(a, P, grant). Performs an external action through a capability exit, such as broker order signing, API write, email send, or deployment.

Commit(R). Records refs, receipts, commitments, and state roots in STATELOG.

Restore($\text{archive}, P$). Rehydrates workspace state after authority checks, hash verification, decryption, and state-root validation.

6 Leakage-Typed cTEE-IR

The compiler is the enforcement point. A cTEE implementation should not rely on operator authors to remember which data may touch an untrusted node. It should compile agent work into a leakage-typed intermediate representation whose typing rules reject accidental plaintext custody before execution.

6.1 Sensitivity Lattice

Let sensitivity labels form a partial order:

$$\text{public} \sqsubseteq \text{redacted} \sqsubseteq \text{sealed} \sqsubseteq \text{guardian} \sqsubseteq \text{authority}.$$

The order is not a disclosure permission. It is a custody boundary. Public and approved redacted values may be materialized on NODE. Sealed values may appear only as ciphertexts, commitments, shares, or handles. Guardian values may be materialized only inside GUARDIAN or a threshold policy path. Authority values may never be handed to NODE; they can only be exercised through capability exits.

Each expression has a type and label:

$$\Gamma \vdash e : \tau^\ell, \quad \ell \in \{\text{public}, \text{redacted}, \text{sealed}, \text{guardian}, \text{authority}\}.$$

The label of a derived expression is at least the join of its dependencies:

$$\ell(e_1 \circ e_2) \supseteq \ell(e_1) \sqcup \ell(e_2).$$

6.2 Remote Admissibility

An operator op may be scheduled on NODE only if the compiler can derive:

$$\Gamma, P \vdash op(e_1, \dots, e_n) \Downarrow_{\text{Node}} (y, R, \mathcal{L}_{op})$$

subject to one of the following admissibility rules:

1. **Plain remote rule.** Every input label is **public** or policy-approved **redacted**, and the output is not more sensitive than policy permits.
2. **Cryptographic remote rule.** Any sealed input is represented on NODE only as FHE ciphertext, MPC share, garbled-circuit wire label, ORAM handle, encrypted blob ref, or commitment. The operator family must match the representation.
3. **Guardian-mediated rule.** The node may issue a bounded query to GUARDIAN and receive only an approved response class: selection bit, masked score, redacted critique, encrypted result, or declassification denial.
4. **Capability-exit rule.** Authority-labeled values cannot be read. The node may propose an action; GUARDIAN, the authority plane, or the execution boundary decides whether to execute it through a scoped capability exit.

If no rule applies, the compiler must reject the remote plan or repartition the task. This is the practical meaning of no-plaintext custody.

6.3 Receipt-Carrying Types

Remote execution returns values with receipt obligations:

$$\Gamma, P \vdash op : \tau_1^{\ell_1} \times \dots \times \tau_n^{\ell_n} \rightarrow \tau_o^{\ell_o} [\mathcal{R}, \mathcal{L}_{op}].$$

Here $\mathcal{R}$ is the set of required receipts, for example:

- `PrivateInferenceReceipt`;
- `ArtifactRecordedReceipt`;
- `DeclassificationReceipt`;
- `CapabilityExitReceipt`.

A value whose receipt obligations are unsatisfied cannot be committed as admitted workspace truth.

6.4 Compiler Invariant

Invariant 1 (No Plaintext by Construction). *If the cTEE compiler accepts a remote plan for NODE, then every sealed, guardian, or authority dependency in that plan is either absent from the node view, represented by an approved cryptographic carrier, or exercised through an approved capability exit.*

7 Core Objects

7.1 PrivateWorkspace

```
PrivateWorkspace:
  workspace_id: workspace://...
  owner_ref: principal://...
  policy_hash: sha256:...
  state_root: sha256:...
  public_trunk_refs: [artifact://...]
  encrypted_object_refs: [artifact://...]
  redacted_projection_refs: [projection://...]
  guardian_ref: guardian://...
  leakage_profile_ref: leakage://...
  receipt_refs: [receipt://...]
```

7.2 PrivateWorkspaceCapsule

```
PrivateWorkspaceCapsule:
  capsule_id: private_workspace_capsule://...
  task_id: task://...
  node_id: runtime://...
  visible_context:
    - public_dataset_ref
    - redacted_summary
    - public_schema
    - bounded_objective
  protected_context:
    - encrypted_payload_ref: artifact://...
    - alpha_seal_ref: alpha_seal://...
    - private_memory_ref: memory://...
  allowed_remote_ops:
    - public_model_inference
    - tensor_compute
    - masked_score_eval
    - encrypted_checkpoint_write
  forbidden_remote_ops:
    - plaintext_secret_read
    - broker_key_read
    - live_portfolio_plaintext_read
    - unbounded_external_action
  leakage_profile:
    visible: [task_type, tensor_dimensions, runtime_cost]
    padded: [output_size, schedule_window]
  receipts:
    required: [private_inference, artifact_recorded, capability_exit_if_action]
```

7.3 AlphaSeal

AlphaSeal is the sealed private-head pattern. It is named for quantitative strategies, but applies to any high-value private decision logic.

```
AlphaSeal:
  alpha_seal_id: alpha_seal://...
  owner_ref: principal://...
  strategy_commitment: sha256:...
  public_trunk_ref: model://... | workflow://...
  private_head:
    representation: local_only | masked_linear | fhe_score |
                    garbled_rules | mpc_share | committed_witness
    encrypted_payload_ref: artifact://...
    leakage_profile_ref: leakage://...
  policy:
    max_notional: ...
    max_daily_loss: ...
    allowed_markets: [...]
    allowed_brokers: [...]
  authority:
    authority_grant_ref: grant://...
    guardian_ref: guardian://...
```

7.4 AutonomyLease

An AutonomyLease lets a private workspace node keep working while the user is offline, without granting plaintext custody:

```
AutonomyLease:
  lease_id: autonomy_lease://...
  owner_ref: principal://...
  node_ref: runtime://...
  valid_until: timestamp
  allowed_without_user_online:
    - refresh_public_data
    - run_public_model_inference
    - update_encrypted_state
    - evaluate_alpha_seal
    - propose_order_intent
    - draft_redacted_report
  forbidden_without_step_up:
    - reveal_strategy_source
    - reveal_pii
    - widen_broker_scope
    - exceed_risk_limits
    - export_private_memory
  guardian_policy:
    required: true
    quorum: 1-of-1 | 2-of-3 | policy_defined
```

7.5 NodeMeasurementReceipt

Consumer GPUs such as rented RTX 3090 nodes generally do not provide the same hardware confidential-computing story as newer attested accelerator stacks. Still, a cTEE can use one-time or periodic node measurement for integrity, metering, and reproducibility. Measurement does not make plaintext safe on the node. It says which public runtime was launched and which public artifacts were bound to the run.

```
NodeMeasurementReceipt:
  receipt_id: receipt://...
  node_ref: runtime://...
  boot_epoch: integer
  runtime_manifest_hash: sha256:...
  container_or_vm_image_hash: sha256:...
  driver_profile_hash: sha256:...
  gpu_profile:
    model: RTX_3090 | H100 | ...
    memory_gb: ...
    cuda_profile_hash: sha256:...
  public_model_hashes: [sha256:...]
  public_trunk_root: sha256:...
  forbidden_plaintext_classes: [...]
  leakage_profile_hash: sha256:...
  measurement_method:
    reproducible_boot | provider_attestation | tee_attestation |
    remote_challenge | policy_declared
  signature: ...
```

This receipt helps answer: did the node run the expected public trunk, model, and operator manifest? It does not answer: can the provider see normal plaintext memory? The cTEE privacy answer remains no-plaintext custody.

8 Private Workspace File System

8.1 Encrypted Names and Patches

Filenames can leak alpha. A path such as:

```
/strategies/smallcap-biotech-post-fda-momentum/
```

may reveal more than the file body. A private workspace therefore supports encrypted names:

$$nameId = H(K_{name} \parallel path \parallel epoch),$$

and stores the display name inside an encrypted metadata object. The node may see:

```
/workspace/public_trunk/
/sealed/alpha_seal://...
/objects/sha256/3f/...
```

while the user sees normal files and folders.

User edits are represented as encrypted patches:

$$\Delta_C \leftarrow \text{AEAD.Enc}_{K_W}(N, A, \langle \text{objectId}, \text{parentHead}, \text{patch}, \text{editor}, \text{policyHash} \rangle).$$

The state log commits:

$$\text{head}' = \text{H}(\text{head} \parallel \text{H}(\Delta_C) \parallel \text{policyHash} \parallel \text{receiptRoot}).$$

8.2 Public Trunk and Private Head

Every private workspace object is classified:

$$\text{role}(o) \in \{\text{publicTrunk}, \text{redactedStub}, \text{sealedPrivate}, \text{guardianOnly}, \text{actionAuthority}\}.$$

Only `publicTrunk` and approved `redactedStub` objects may be mounted as plaintext on NODE. A public trunk should contain generic pipelines, schemas, backtest engines, public data transforms, report templates, and public model routes. A private head contains alpha formulas, coefficients, thresholds, risk gates, live portfolio, private memory, broker keys, and final action logic.

9 Plaintext-Free Runtime Mounting

Plaintext-Free Runtime Mounting is the runtime interface that makes cTEE useful for ordinary tools, files, shells, model servers, and model calls. A runtime mount is not a decrypted filesystem mount. It is a typed projection of a private workspace into a runtime view:

$$\text{MountView} = \langle S_{\text{pub}}, \text{Redact}(S_{\text{red}}), \text{Ref}(\text{Enc}(S_{\text{seal}})), \text{Handle}(S_{\text{guard}}), \text{Exit}(S_{\text{auth}}) \rangle.$$

Tools and models may consume public content, redacted projections, encrypted refs, candidate commitments, private-head handles, declassification request handles, and capability-exit handles. They must not receive protected files, prompts, memory, strategy source, credentials, or private authority as provider-readable plaintext. Plaintext-Free Model Mounting is the model-facing specialization of this broader runtime mount.

```
PlaintextFreeModelMount:
  mount_id: model_mount://...
  workspace_id: workspace://...
  node_ref: runtime://...
  visible_entries:
    - public_file
    - public_dataset
    - redacted_projection
  protected_entries:
    - encrypted_workspace_object_ref
    - private_head_handle
    - declassification_request_handle
    - capability_exit_handle
  forbidden_plaintext_classes:
    - pii
    - strategy_source
    - private_memory
    - broker_secret
    - live_portfolio
  leakage_profile_ref: leakage://...
```

```
deterrence_detection_profile_ref: deterrence://...
receipts_required:
  - ModelMountReceipt
  - PrivateInferenceReceipt
  - LeakageReceipt
```

This is the precise role of ciphering in cTEEs. Ciphering is useful only when the node never receives the deciphering key and never materializes protected plaintext or plaintext-equivalent private activations. The model mount prevents accidental custody: the remote model can reason over what it is allowed to see, and must ask private handles for anything else.

Runtime-mount doctrine. Tools and models may operate against the workspace. The node must not become the plaintext custody domain for protected workspace state.

10 Deterrence and Detection

Deterrence and detection are not privacy primitives. They are attribution, abuse-response, and dispute primitives. They matter because rented nodes, marketplace workers, and service providers can still deny service, replay work, copy public/redacted outputs, or attempt to lure users into unsafe plaintext mounts.

A high-value cTEE workspace may bind every capsule and model mount to a deterrence profile:

```
DeterrenceDetectionProfile:
  profile_id: deterrence://...
  provider_bound_nonce: sha256:...
  node_bound_watermark: sha256:...
  canaries:
    - decoy_strategy
    - decoy_credential
    - synthetic_pii
    - watermark_phrase
    - honeypoint_endpoint
  watermarking:
    candidate_lattice_watermark: true
    report_text_watermark: true
    code_patch_watermark: true
  monitoring:
    scan_public_leaks: true
    broker_honeypoint_alerts: true
    suspicious_replay_detection: true
```

Canaries must be synthetic. They must not be real secrets, and they must be labeled so they do not contaminate training, memory, settlement, or user decisions. Watermarks must identify a capsule, provider, node, or run without revealing private strategy content.

```
DeterrenceDetectionReceipt:
  receipt_id: receipt://...
  profile_ref: deterrence://...
  event_type:
    canary_planted | watermark_bound | honeypoint_bound |
    canary_tripped | suspicious_replay_detected | leak_scan_completed
```



```

bound_refs:
  - model_mount://...
  - private_workspace_capsule://...
public_evidence_refs:
  - artifact://...
private_evidence_commitments:
  - commitment://...
action:
  none | warn_user | revoke_node | rotate_keys |
  open_dispute | slash_provider | quarantine_workspace

```

This layer makes cTEE better suited to marketplaces such as service outcome networks and worker catalogs: a dispute can show that a provider-bound canary, watermark, or replay signature escaped without requiring the user to reveal the real private workspace.

11 Protocols

Protocol. Remote Node Enrollment

1. NODE publishes hardware profile, runtime manifest, driver profile, public model hashes, supported operator families, and storage policy.
2. GUARDIAN or the coordinator challenges NODE to produce a `NodeMeasurementReceipt`.
3. STATELOG records the measurement receipt, node policy, revocation epoch, and accepted leakage profile.
4. The scheduler may route public trunk work to NODE only while the measurement and lease remain valid.
5. Protected workspace classes remain forbidden regardless of measurement quality unless a stronger policy explicitly enables a hardware TEE mode.

Protocol. Open Private Workspace

1. USER selects a runtime node and a privacy policy P .
2. GUARDIAN derives or authorizes workspace keys K_W , name keys K_{name} , and declassification policy D .
3. STATELOG creates a workspace genesis operation:

$$G = H(\text{workspaceId}, \text{owner}, \text{policyHash}, \text{guardianRef}, \text{schemaVersion}).$$

4. BLOBSTORE receives encrypted root objects and sealed archive refs.
5. NODE receives the public trunk, redacted projections, encrypted refs, and allowed operator manifest.
6. The user sees a normal workspace view. The node sees no protected file plaintext by default.

Protocol. Compile Private Workspace Capsule

1. Classify task inputs by sensitivity class.
2. Build visible context from public trunk and redacted projections.
3. Replace protected references with encrypted artifact refs, commitments, sealed private heads, or guardian calls.
4. Attach forbidden plaintext classes and allowed operator families.
5. Attach a leakage profile $\mathcal{L}$ and receipt requirements.
6. Commit the capsule hash and policy hash in STATELOG.

Protocol. Declassification Gate

1. Protected output Y_C or commitment C_Y is produced.
2. NODE submits a **PrivateInferenceReceipt**.
3. GUARDIAN checks policy, autonomy lease, revocation epoch, leakage profile, and requested disclosure class.
4. If allowed, GUARDIAN decrypts or reconstructs Y , scans it, and emits a declassification receipt.
5. If the output implies an external action, the action is routed through a **CapabilityExit**. The node never receives durable raw credentials.

12 Security Definitions

12.1 No-Plaintext-Custody

Definition 1 (No-Plaintext-Custody). A cTEE scheme satisfies *no-plaintext-custody (NPC)* for protected class set $\mathcal{C}$ and leakage function $\mathcal{L}$ if for every probabilistic polynomial-time adversary $\mathcal{A}$ controlling NODE, there exists a simulator $\mathcal{S}$ such that for any two workspaces W_0, W_1 with $\mathcal{L}(W_0) = \mathcal{L}(W_1)$, the adversary’s view during node execution is computationally indistinguishable from $\mathcal{S}(\mathcal{L}(W_b))$, excluding values explicitly declassified by policy.

$$|\Pr[\mathcal{A}(\text{View}_{\text{Node}}(W_0)) = 1] - \Pr[\mathcal{A}(\text{View}_{\text{Node}}(W_1)) = 1]| \leq \text{negl}(\lambda).$$

NPC is not the same as hiding all metadata. It hides protected plaintext modulo declared leakage and explicit declassification.

12.2 Authority Safety

Definition 2 (Authority Safety). A cTEE satisfies *authority safety* if a node-controlled adversary cannot produce an accepted external effect a unless there exists an unexpired authority grant g , a policy P , and a valid receipt chain proving:

$$\text{allowed}(P, g, a) = \text{true}.$$

External effects include broker orders, payments, email sends, API writes, deployments, secret exports, and persistent state imports.

12.3 Receipt Binding

Definition 3 (Receipt Binding). *A receipt R is binding if no efficient adversary can produce two accepted interpretations of the same receipt hash with different actor, policy, authority, input commitment, output commitment, or resulting state root.*

12.4 Restore Validity

Definition 4 (Restore Validity). *A restored workspace state is valid if the restore operation verifies archive hashes, decrypts under authorized policy, checks object heads, checks state root, records import receipts, and rebuilds projections from accepted refs rather than silently mutating local files.*

13 Security Theorems

Assumption 1 (Primitive Security). *AEAD is IND-CCA and INT-CTXT secure; commitments are binding and hiding; signatures are existentially unforgeable; hash functions are collision resistant; guardian thresholds satisfy their stated corruption bounds; and FHE/MPC/GC/ORAM operators satisfy their standard security definitions when used.*

Assumption 2 (No Accidental Plaintext). *The implementation enforces the capsule policy: protected classes are not mounted, logged, prompted, cached, traced, or sent to NODE as normal plaintext unless a declassification receipt explicitly allows it.*

Theorem 1 (NPC Security of Base cTEE). *Under the primitive security and no-accidental-plaintext assumptions, the base cTEE construction satisfies no-plaintext-custody for protected workspace objects whose plaintext never leaves USER or GUARDIAN.*

Proof sketch. The node receives public trunk objects, redacted projections, encrypted payloads, commitments, receipts, and declared leakage. By AEAD security, sealed workspace objects are indistinguishable from random strings to a node lacking keys. By commitment hiding, committed private values reveal only declared commitment metadata. By the no-accidental-plaintext assumption, no alternate path gives the node protected plaintext through logs, prompts, mounts, caches, or traces. Therefore the node view can be simulated from public trunks, redacted projections, ciphertext lengths, commitments, receipts, and leakage $\mathcal{L}$. $\square$

Theorem 2 (Authority Safety of Capability Exits). *If signature unforgeability holds and the authority verifier accepts only unexpired grants satisfying policy predicates, then a node adversary cannot cause an accepted external effect outside the granted scope except with negligible probability.*

Proof sketch. Every external effect requires a capability-exit receipt bound to a grant, policy hash, action summary, input commitment, output commitment, and signature or threshold approval. Producing an accepted effect outside policy requires either forging an authority signature, finding a binding collision in the receipt, or exploiting a verifier bug outside the model. $\square$

Proposition 1 (No Free Full-Speed Private LLM Inference). *On a root-controlled consumer GPU without trusted hardware, trusted guardian participation, or cryptographic private evaluation, arbitrary private LLM inference over protected plaintext at ordinary plaintext token speed cannot be claimed by a software wrapper alone.*

Argument. If the node performs the same plaintext matrix and attention operations as a normal LLM over private tokens or embeddings, then intermediate plaintext or semantically equivalent representations must exist in node-observable process memory, logs, IPC, kernel buffers, driver buffers, or GPU-visible memory under the adversary model. A software wrapper that leaves those representations on the node changes neither the information flow nor the provider’s root access. To remove the plaintext from the node, the computation must move to another trust domain, use hardware isolation, or use cryptographic representations with nonzero overhead. Thus a pure software wrapper cannot honestly claim arbitrary private LLM inference at normal plaintext speed on the adversarial node. $\square$

14 Candidate-Lattice Private Decoding

The preceding proposition is negative but not fatal. Many private agent tasks do not require the untrusted node to see private context. They require the node to produce useful public reasoning candidates, while private state selects, filters, reranks, or authorizes them. Candidate-Lattice Private Decoding (CLPD) is therefore the default cTEE protected-agency strategy.

14.1 Construction

Let M_{pub} be a public model running on NODE. Let h_{priv} be a private scoring head held by GUARDIAN, MPC, FHE, or a local client. At step t :

1. NODE expands a candidate lattice:

$$\mathcal{Y}_t = \{y_{t:t+B}^{(1)}, \dots, y_{t:t+B}^{(K)}\} \leftarrow M_{\text{pub}}(x_{\text{pub}}, y_{<t}).$$

2. NODE commits to the lattice:

$$C_t = H(\mathcal{Y}_t \parallel \text{modelHash} \parallel \text{policyHash} \parallel \text{nonce}).$$

3. GUARDIAN evaluates private utility:

$$s_i = h_{\text{priv}}(y_{t:t+B}^{(i)}, S_{\text{priv}}, P).$$

4. GUARDIAN returns either an encrypted selection, a masked reranking, a redacted critique, or a bounded next prefix. The node receives no private state.
5. Receipts bind C_t , the selected branch commitment, leakage profile, and declassification decision.

14.2 Why It Matters

CLPD preserves the high-throughput path for public transformer work. The GPU does what it is good at: generate candidate continuations, embeddings, public summaries, report drafts, code skeletons, or strategy simulations. The private head does what must remain private: choose among candidates using PII, private policy, live portfolio, alpha formula, or user secrets.

The user-facing product should not expose CLPD as a selectable mode. The user opens a private workspace. The runtime uses CLPD by default wherever protected agency is required and the public/generic candidate generation can be separated from private selection.

CLPD does not implement arbitrary private inference. It implements the default useful class of private agency:

$$\text{public generation} + \text{private selection} + \text{receipted action}.$$

14.3 Security

If the candidate lattice is generated from public or redacted context and the private head reveals only the selected branch under a leakage profile, then the node learns at most the public candidates, timing, output length, and the declared selection leakage. Padding, delayed reveal, top- m selection, batching, and decoy selections can reduce selection leakage.

14.4 Selection Leakage Accounting

CLPD should attach a leakage account to every private selection. A single chosen branch from K candidates leaks at most:

$$L_{\text{select}} \leq \log_2 K$$

bits about the private preference, before considering side information. These bounds account for explicit selection observables, not all semantic inference an adversary may derive from the public candidate set, timing, prior runs, or domain-specific side information. Revealing an unordered top- m set leaks at most:

$$L_{\text{top}m} \leq \log_2 \binom{K}{m}.$$

Revealing a total ranking leaks at most:

$$L_{\text{rank}} \leq \log_2(K!).$$

The receipt should also account for timing, size, and schedule leakage. For a bucketed observable X with b_X possible buckets, use the conservative bound:

$$L_X \leq \log_2 b_X.$$

A practical `LeakageBudget` can therefore be represented as:

```
LeakageBudget:
  budget_id: leakage://...
  task_class: quant_backtest | private_email | legal_review | ...
  candidate_count_k: integer
  reveal_policy: selected_one | top_m | masked_score | denial_only
  selection_bits_bound: number
  timing_bucket_bits_bound: number
  size_bucket_bits_bound: number
  schedule_bucket_bits_bound: number
  cumulative_budget_bits: number
  mitigations:
    - fixed_k
    - top_m_or_decoy_reveal
    - delayed_commit_reveal
    - batch_padding
    - schedule_jitter
    - neutral_labels
```

This is intentionally conservative. It does not prove semantic secrecy by itself; it makes leakage explicit enough for product policy. A high-value quant workspace may set a low daily leakage budget, require decoy reveals, and stop or escalate once cumulative budget is exhausted.

14.5 Agent Use Cases

- **Quant.** The node generates candidate trade narratives or factor sets; the private alpha head scores them with sealed coefficients and live portfolio constraints.
- **Legal.** The node drafts public clause alternatives; a private policy head selects clauses compatible with confidential negotiation strategy.
- **Personal assistant.** The node drafts generic responses; the guardian filters against private user history and declassifies only approved text.
- **Code agent.** The node proposes public trunk patches; private workspace objects stay local, and sensitive diffs are applied through encrypted patches.

15 Default cTEE Agent Execution

A private agent harness using cTEE should route every operation by sensitivity, with CLPD as the default protected-agency path:

Operation	Allowed on node	Protected path
Model mount	Public/redacted view and private handles	ModelMountReceipt
Public LLM inference	Plaintext public prompt	None required
Redacted summary	Redacted projection	Redaction receipt
Private file edit	Encrypted patch only	Browser/device/guardian view
Private retrieval	Encrypted refs or redacted snippets	Local/guardian/ORAM path
Protected agency	Candidate lattice on node	AlphaSeal/guardian selection
Private scoring	Masked, FHE, MPC, GC, or committed witness	Guardian/private head
External action	Proposal only	Capability exit
Restore/import	Encrypted archive fetch	Authority and state-root checks

16 Quantitative Strategy Pattern

For the reference quant workflow, the private workspace separates:

$$\text{Strategy} = \text{PublicTrunk} \parallel \text{AlphaSeal}.$$

PublicTrunk may include data loaders, generic factor libraries, backtest engines, public market data transforms, report templates, simulation kernels, and public model routes. **AlphaSeal** contains signal formulas, private coefficients, thresholds, risk gates, live portfolio constraints, broker keys, and final order logic.

The rented GPU can run:

- market data ingestion;
- public feature tensor generation;
- public model inference;
- public or redacted research;

- public backtests;
- encrypted checkpointing;
- candidate ranking under masked or committed heads;
- report drafting over redacted observations.

The rented GPU must not receive:

- strategy source plaintext;
- private coefficients and thresholds;
- broker credentials;
- live portfolio plaintext;
- final order logic;
- unrestricted signing authority.

16.1 End-to-End Quant Walkthrough

A private quant workspace can still feel like normal remote development:

1. The user opens `signals.py` in the browser, local agent harness, or CLI authority view. The plaintext file is decrypted there, not in the rented node's workspace.
2. The edit is saved as an encrypted patch with neutral object identifiers and committed to the state log with policy, parent head, and receipt refs.
3. The rented GPU node receives the public trunk, public market-data tensors, redacted projections, encrypted refs, and the allowed operator manifest.
4. The node runs public backtests, feature generation, model inference, and candidate trade or report generation at ordinary GPU speed.
5. `AlphaSeal` scores, filters, or reranks candidate trades through the guardian, threshold path, masked head, or cryptographic operator. The node sees only the allowed response class and leakage receipt.
6. The authority plane signs, denies, or escalates an order intent through a configured capability exit. Broker keys and unrestricted order logic are never handed to the node.
7. The state log records the encrypted patch, artifact refs, candidate commitment, leakage receipt, declassification receipt, and capability-exit receipt.

The product promise is simple: the user edits and runs a private workspace; the provider sees compute jobs, public trunks, ciphertexts, commitments, receipts, and bounded disclosures.

17 Leakage Profiles

A cTEE cannot hide everything. It must declare leakage:

$$\mathcal{L} = \langle \text{taskType}, \text{modelFamily}, \text{publicDataset}, \text{tensorShape}, \\ \text{jobDuration}, \text{outputLength}, \text{schedulePattern}, \\ \text{storageSize}, \text{receiptTypes} \rangle.$$

Mitigations include:

- neutral names for paths and jobs;
- encrypted file and folder names;
- batching and schedule jitter;
- padded output sizes;
- decoy jobs for high-value workflows;
- redacted trace and prompt policies;
- ORAM or private lookup for access-pattern-sensitive workflows.

The leakage profile is a first-class object because it is the line between a credible privacy claim and marketing fog.

17.1 Leakage Receipts

Every CLPD selection should emit a leakage receipt. An implementation may also include the same leakage fields in the `PrivateInferenceReceipt`:

```
LeakageReceipt:
  receipt_id: receipt://...
  capsule_id: private_workspace_capsule://...
  candidate_lattice_commitment: commitment://...
  leakage_profile_ref: leakage://...
  selection_policy: selected_one | top_m | denial_only | masked_score
  selection_bits_bound: number
  timing_bucket_bits_bound: number
  size_bucket_bits_bound: number
  cumulative_budget_before: number
  cumulative_budget_after: number
  mitigation_refs:
    - padding_policy://...
    - schedule_jitter_policy://...
    - decoy_policy://...
```

The purpose is operational: a user, service contract, or verifier can decide whether continued private work is still within the promised leakage envelope.

18 Implementation Guidance

18.1 Harness Integration

Any agent harness can implement cTEE:

- route work by sensitivity before tool/model execution;
- store private workspace objects as encrypted refs;
- send only public trunks, redacted projections, commitments, or sealed capsules to untrusted nodes;
- bind every protected output to a receipt;
- route external effects through capability exits;
- make declassification explicit in UI and logs;
- make unsafe plaintext mounts visible and hard to choose.

The concept is not tied to one product. A private agent harness, a marketplace worker runtime, an enterprise runner, or an open-source autonomous-code agent can adopt the same envelope.

18.2 Minimum Viable cTEE

The smallest useful implementation has:

1. encrypted workspace object store with AEAD and state-root binding;
2. encrypted or neutral filenames for sensitive paths;
3. public trunk/private head classification;
4. Plaintext-Free Runtime Mounting with `ModelMountReceipt` for model views;
5. private workspace capsule compiler;
6. Candidate-Lattice Private Decoding for protected agency by default;
7. guardian or client-side declassification;
8. capability exit for secrets and external actions;
9. synthetic canaries or provider-bound watermarks for high-value workspaces, recorded through deterrence/detection receipts;
10. receipt log with private inference, declassification, artifact, and action receipts.

18.3 Conformance Tests

1. A protected file never appears as plaintext on NODE during normal open, edit, run, checkpoint, trace, or restore.
2. A protected filename does not appear in node-visible paths in high security mode.
3. A model prompt containing PII is blocked, redacted, or rerouted.
4. A private strategy head can be evaluated without revealing source to NODE.
5. A broker action cannot be executed without a valid capability exit.
6. A restore cannot silently mutate local files without state-root and authority receipts.
7. A receipt proves private work without revealing protected plaintext.

19 Relation to Existing Work

cTEEs are not a replacement for FHE, MPC, ORAM, garbled circuits, zero knowledge, or hardware confidential computing. They are a composition layer that chooses among them while preserving an agent-facing workspace abstraction.

FHE provides the strongest single-node cryptographic privacy story for functions that can be evaluated efficiently over ciphertext. MPC often provides more practical performance with a committee trust model. Garbled circuits fit boolean private logic and policy checks. ORAM mitigates access-pattern leakage. Hardware TEEs may allow plaintext inside an attested boundary, but introduce hardware trust and side-channel assumptions. cTEEs can use all of these, but their base claim is no-plaintext custody, not universal private computation.

20 Open Problems

1. **Guardian minimization.** Treat the guardian as an authority view, not a new machine: authenticated browser, local agent harness, CLI signer, mobile passkey, enterprise key service, authority network, or threshold policy. Research should shrink this surface to decryption, private-head selection, budget accounting, and capability signing.
2. **Selection leakage.** Turn CLPD leakage into a measurable budget. The immediate path is fixed candidate counts, top- m or decoy reveal, delayed commit-reveal, schedule jitter, size padding, and cumulative leakage receipts.
3. **Private transformer performance.** Reserve FHE/MPC transformer work for narrow private operators and small heads first. Use CLPD for default high-throughput agency until encrypted attention, sampling, and KV caches become practical enough for broader use.
4. **Encrypted developer tools.** Build diff, search, test, lint, and refactor tools that run in the authority view over locally decrypted objects, then commit encrypted patches and redacted summaries to the node.
5. **Private workspace UX.** Make unsafe plaintext mounts impossible by default and visually unmistakable when explicitly chosen. The normal product state should simply say: **Private Workspace On.**

6. **Receipt privacy.** Prove useful work occurred without revealing too much about the work. Near-term receipts should bind candidate lattice commitments, leakage budgets, redaction policy, and declassification outcomes.
7. **Private retrieval.** Combine encrypted names, neutral labels, coarse buckets, ORAM for high-sensitivity indexes, and local/guardian reranking so access patterns do not reveal the private workspace structure.
8. **Correctness for public candidates.** The node may generate poor or adversarial candidates. Use redundant public generation, deterministic replayable candidate commitments, verifier sampling, and dispute-triggered trace disclosure for settlement-grade work.

21 Conclusion

cTEEs make a simple claim technically precise:

Open Private Workspace. Run on untrusted compute. Do not give the node plaintext custody.

The contribution is not that software can magically hide plaintext from root. The contribution is a disciplined architecture for avoiding plaintext custody while preserving useful remote compute: encrypted workspace objects, public trunks, Candidate-Lattice Private Decoding by default for protected agency, sealed private heads, private workspace capsules, declassification gates, authority-bounded capability exits, receipts, and explicit leakage profiles. This is enough to make persistent private agents practical on rented GPU infrastructure without pretending that the infrastructure is trusted.

References

- [1] Craig Gentry. Fully homomorphic encryption using ideal lattices. STOC 2009. <https://dl.acm.org/doi/10.1145/1536414.1536440>
- [2] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. ASIACRYPT 2017. <https://eprint.iacr.org/2016/421>
- [3] Ran Gilad-Bachrach et al. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. ICML 2016. <https://proceedings.mlr.press/v48/gilad-bachrach16.html>
- [4] Meng Hao et al. MPCFormer: Fast, performant and private Transformer inference with MPC. arXiv 2022. <https://arxiv.org/abs/2211.01452>
- [5] Emil Stefanov et al. Path ORAM: An extremely simple oblivious RAM protocol. CCS 2013. <https://eprint.iacr.org/2013/280>
- [6] Andrew C. Yao. How to generate and exchange secrets. FOCS 1986. <https://ieeexplore.ieee.org/document/4568207>
- [7] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game. STOC 1987. <https://dl.acm.org/doi/10.1145/28395.28420>
- [8] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. CRYPTO 1991. https://link.springer.com/chapter/10.1007/3-540-46766-1_9
- [9] NVIDIA. Confidential computing on H100 GPUs for secure and trustworthy AI. <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>
- [10] AMD. AMD confidential computing. <https://www.amd.com/en/products/processors/server/epyc/confidential-computing.html>
- [11] Intel. Intel Trust Domain Extensions security research and assurance. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/tdx-security-research-and-assurance.html>
- [12] Zcash Foundation. Zcash protocol specification. <https://zips.z.cash/protocol/protocol.pdf>